Assignment 03

Published | Assign To | Edit

Use of Generative AI Is Not Permitted

Learning Objectives

This assignment's goal is to add automated integration tests for the web application APIs and implement continuous integration with GitHub Actions. By completing this assignment, you will:

- Understand the importance of integration testing in API development
- Learn to write effective API tests using REST Assured
- Practice testing various HTTP methods and status codes
- Implement data validation and error handling tests
- Work with test data management and cleanup
- Apply best practices for API test automation
- Understand the fundamentals of Continuous Integration
- Learn how to create and configure GitHub Actions workflows
- Implement automated testing in a CI pipeline
- Configure branch protection rules to enforce quality gates
- Gain hands-on experience with modern DevOps practices

Integration Tests

You are required to implement comprehensive integration tests for an API-only backend web application using REST Assured or a similar testing library. This assignment will test your understanding of API testing principles, test automation, and quality assurance practices.

1. Technology Stack

Primary Testing Library (Choose One of the following or any thing similar):

- REST Assured (Java) - Recommended
- Postman/Newman (JavaScript/Collection-based)
- Requests + Pytest (Python)
- SuperTest (Node.js/JavaScript)

Additional Tools:

- Build tool (Maven/Gradle for Java, npm for Node.js, pip for Python)

- Test runner (JUnit/TestNG for Java, Jest for Node.js, Pytest for Python)
- Assertion libraries as needed

2. Target API Specifications

Your tests should cover all RESTful APIs we have built so far and can be found at **https://app.swaggerhub.com/apis-docs/csye6225-webapp/cloud-native-webapp/fall2025-a2 (https://app.swaggerhub.com/apis-docs/csye6225-webapp/cloud-native-webapp/fall2025-a2)**

Each test should validate:

- **HTTP Status Code**: Correct status codes (200, 201, 400, 401, 404, etc.)
- **Response Headers**: Content-Type, authentication headers, etc.
- **Response Body**: JSON structure, field values, data types
- **Side Effects**: Data persistence, state changes, etc.

3. Required Test Categories

A. Positive Test Cases

- **Creation Tests**
  - Test successful user creation with valid data
  - Test successful product creation with valid data
  - Verify response status code (201)
  - Validate response body structure and data
  - Test user creation with different valid input combinations
- **Retrieval Tests**
  - Test getting product
  - Test getting user by valid ID
  - Test getting user
  - Test getting product by valid ID
  - etc.
- **Update Tests**
  - Test successful updates for supported endpoints
  - Verify partial updates work correctly
  - Test updating different fields independently
- **Authentication Tests**
  - Test successful login with valid credentials
  - Test successful registration
  - Test accessing protected endpoints with valid token

B. Negative Test Cases

- **Invalid Input Tests**
  - Test creation with missing required fields
  - Test user creation with invalid email format
  - Test user creation with duplicate email/username
  - Test updates with invalid data types
  - etc.
- **Authentication Error Tests**
  - Test login with invalid credentials
  - Test accessing protected endpoints without token
  - etc.
- **Resource Not Found Tests**
  - Test getting non-existent user by ID
  - Test updating non-existent user
  - Test getting non-existent product by ID
  - Test updating non-existent product
  - etc.
- **HTTP Method Tests**
  - Test using wrong HTTP methods on endpoints
  - Test unsupported endpoints (404 errors)

C. Edge Case Tests

- **Boundary Value Tests**
  - Test with maximum/minimum string lengths
  - Test with maximum/minimum numeric values
  - Test with special characters in input
- **Performance/Load Tests (Basic)**
  - Test response times are reasonable
  - Test concurrent requests handling
  - Test with large datasets
- **Data Integrity Tests**
  - Test that created data persists correctly
  - Test that updates don't affect unmodified fields
  - Test that deleted data is properly removed

Continuous Integration w/GitHub Actions

Create GitHub Actions Workflow

Add a GitHub Actions workflow that automatically runs tests for each pull request.

1. Create a `.github/workflows/` directory in your repository root
2. Create a workflow file (e.g., `ci.yml` or `test.yml`) with the following specifications:
   - Trigger on pull requests to the `main` branch
   - Run on the latest version of Ubuntu
   - Support your project's programming language and version
   - Install project dependencies
   - Execute all tests
   - Fail the workflow if any tests fail

Implement Branch Protection Rules

Configure **GitHub branch protection rules (http://Managing%20a%20branch%20protection%20rule)** for the `main` branch of your organization repository with:

- Require a pull request before merging
- Require status checks to pass before merging
- Require branches to be up to date before merging
- Include administrators in restrictions
- Allow force pushes (disabled)
- Allow deletions (disabled)

Assignment Submission Instructions

Development Workflow- Branch Management

- Complete all work on a feature branch in your forked repository
- Merge your feature branch to `main` only when development is complete
- Ensure your feature branch and `main` branch are synchronized before submission

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components

- Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input
  - `account_updated` field: Set automatically by application, never by user input
  - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Grading Guidelines for Git Workflow & Repository Management

Repository Setup & Access

- **Fork Verification**: Confirm the student has created a proper fork of the organization's repository
- **Collaborator Access**: Verify TAs have been added as collaborators with appropriate permissions
- **Repository Location**: Ensure the dev environment is **not** set up in `Downloads` folder or other inappropriate locations

Git Protocol & Configuration

- **SSH Protocol**: Verify the repository was cloned using `git/ssh` protocol (not `https`)
  - **Verification method**: Run `git remote -v` in the cloned repository
  - **Expected output**: URLs should begin with `git@github.com:` not `https://`
- **Remote Configuration**: Ensure the upstream remote is properly configured for the original organization repository

Branch Management & Workflow

- **Main Branch Sync**: Verify `main` branch contains all assignment code and is properly synchronized

  - **Verification method**: Have students create pull requests between `main` and assignment branches
  - **Expected result**: No uncommitted changes should exist
- **Commit History**: Confirm no direct commits were made to the organization repository (all work done on a fork)
- **Branch Strategy**: Verify proper use of feature branches if required by assignment specifications

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories
- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Oct 2 at 3pm	Everyone	Sep 25 at 6pm	Oct 3 at 3pm

+ Rubric